

Honest Assessment

The Good News First

This plan is **significantly above what most first-time fraud detection prototypes look like**. The architecture is sound, the layered approach (rules → scoring → network) is exactly how production fraud systems work, and you clearly extracted a lot from a 2-hour KT with bad notes.

What's genuinely strong:

- Three-layer intelligence pipeline is industry-standard architecture
- Document fraud checklist is specific and actionable
- The false positive scenarios show maturity — most people forget this
- Domain-agnostic design is smart product thinking
- The demo script is compelling and tells the right story

Now the Realistic Part

Rating: 7/10 as a prototype plan, 4/10 as a production fraud system

That gap isn't a criticism — it's the nature of prototypes. But you need to understand where the gap is so you don't oversell.

What This Plan Actually Is vs. What It Appears To Be

What it appears to be: An intelligent fraud detection system that catches fraud before auto-adjudication.

What it actually is: A well-structured demo environment with synthetic data, deterministic rules, and an LLM wrapper that presents pre-embedded fraud scenarios convincingly.

That's not a bad thing for a prototype. But be honest with yourself about it.

Critical Gaps You Should Know About

1. The Synthetic Data Problem (Biggest Risk)

Your entire system runs on data you generate. Every fraud scenario you "catch" is one you planted. This is fine for a demo, but:

- You have **zero validated fraud patterns** from Prudential's actual data
- Your 10 fraud scenarios are educated guesses based on a 2-hour conversation
- The risk scores are circular — you set the flags, then you detect the flags
- A skeptical technical evaluator at Prudential will see this immediately

What industry systems have that you don't: Months or years of labeled historical data, actuarial models, and feedback loops from investigators confirming or dismissing alerts.

Mitigation: Be upfront. Frame it as "this is the detection framework — plug in real data and the same architecture works." Don't pretend the synthetic results prove anything.

2. The Document Analysis Gap (You're Faking the Hard Part)

Your document analysis engine is **metadata flags on synthetic data**. The actual hard problem — detecting font inconsistencies, erasures, typed-over-handwritten text in real PDFs — is one of the hardest problems in document forensics.

Real document fraud detection requires:

- OCR with font extraction (not trivial)
- Image forensics for erasure/alteration detection
- Signature verification models trained on specimens
- PDF metadata parsing and layer analysis

You're not doing any of this. You're saying "this claim has `erasure_indicators: true`" in your synthetic data and then reporting it.

This is the most oversold part of your plan. If Prudential's current process is manual medical document inspection, they'll be very interested in automation — and very disappointed if they realize it's simulated.

Mitigation: Be explicit that document analysis is simulated in the prototype. Show the framework for what checks would run, but don't imply you're actually analyzing documents.

3. The "35 Dependents" Problem

You keep referencing this case because it's dramatic and easy to catch. But:

- This is a **trivially detectable** fraud pattern — a simple SQL query catches it
- If this actually happened and wasn't caught, the problem isn't missing a fraud detection AI — it's that nobody wrote a basic business rule
- Leading your demo with this might make Prudential think "we just need better auto-adjudication rules, not an AI platform"

Better framing: "The 35-dependent case is obvious in hindsight. Our system catches it, yes — but more importantly, it catches the *subtle* patterns: the provider mill with 15 claims from different members, the dependent ring across employers, the tampered medical records. Those are what a simple rule can't find."

4. Rules Without Validation

Your 14 rules are reasonable but **completely unvalidated**:

What you guessed	What you don't know
R-001: >5 dependents in 30 days = suspicious	What's the actual normal rate of dependent additions?
R-003: Claim < 90 days after policy start	What's Prudential's actual early claim distribution?
R-005: Same provider + 5 patients in 30 days	What's normal provider volume for supplemental health?
R-009: Claim = exact benefit max, 3x consecutive	How often does this happen legitimately?

In production fraud systems, every threshold is calibrated against historical data to hit a target false positive rate (typically 2-5%). Your thresholds are guesses.

This is expected for a prototype. Just know it.

5. Network Analysis Sounds Better Than It Is

Your network graph will look great in a demo. But:

- With synthetic data, the rings you detect are rings you planted
- Real network analysis requires entity resolution (matching "John Smith" at "123 Main St" with "J. Smith" at "123 Main Street") — which is its own massive problem
- 200 members and 300 dependents is too small to demonstrate meaningful network patterns
- Real fraud networks emerge from millions of claims over years

6. The Copilot Is a UI Layer, Not Intelligence

Your LangGraph agent calls tools that return synthetic data. The "intelligence" is in the rules engine and risk scoring — the copilot is presentation. That's fine, but know what's doing the work.

The copilot adds value in:

- Making the investigation workflow conversational
- Tracking checklist progress
- Surfacing relevant context without system-switching

The copilot does NOT add value in:

- Actually detecting fraud (that's layers 1-3)
 - Making judgment calls (you correctly say it shouldn't)
 - Replacing analyst expertise
-

Industry Standard Comparison

Capability	Industry Standard	Your Plan	Gap
Rules engine	Hundreds of rules, calibrated quarterly	14 rules, uncalibrated	Large but expected
Risk scoring	ML models on millions of labeled claims	Weighted heuristic	Large but you acknowledge this
Document forensics	Specialized vendors (e.g., ABBYY, own ML models)	Simulated metadata	Very large
Network analysis	Graph databases (Neo4j), billions of edges	NetworkX, 200 nodes	Large but appropriate for prototype
Real-time scoring	Sub-second on live claims	Batch on synthetic data	Expected for prototype
Feedback loop	Analyst actions retrain models	None	Expected for prototype
False positive rate	2-5% measured and managed	Unknown, estimated	Expected
Integration	Connected to claims systems, EHR, payment	Standalone	Expected for prototype

Industry leaders in this space: FICO (Falcon), SAS Fraud Framework, Shift Technology, Friss. These have 10+ years of development, billions of data points, and dedicated data science teams.

You're not competing with them on capability. You're competing on:

- Speed to value (they take 12-18 months to implement)
- Supplemental health specificity (most focus on medical/P&C)
- Analyst experience (most are backend-only, no copilot)
- Cost (they're \$1M+ implementations)

What You Probably Missed From the KT

Based on the gaps in your plan, you likely missed or under-captured:

1. **Auto-adjudication rules specifics** — What exact criteria make a wellness claim auto-pay? This is the attack surface for fraud and you don't have it
2. **Suspicious banner details** — This is their existing flagging system. You need to complement it, not conflict
3. **Actual claim volume and distribution** — Your 500 claims may be wildly off from their real daily volume
4. **Examiner workflow details** — The exact clicks and screens they use. Your UI won't feel real without this
5. **What fraud they've actually seen** — Beyond the 35-dependent case, what patterns have they encountered?
6. **ET rule complexity** — You mention it but your plan's handling is thin. This is apparently a major pain point

My Actual Recommendation

This is a strong prototype plan. Ship it as designed, but adjust your positioning:

~~"We built a fraud detection system"~~

"We built a fraud detection *framework* that demonstrates how rules, risk scoring, network analysis, and analyst workflow can work together on Prudential's supplemental health claims. The synthetic data proves the architecture. Real data proves the value."

The honest pitch is more compelling than the inflated one, because the people at Prudential who evaluate this will know exactly what's real and what's demo. Respect their intelligence, show the architecture, and let the 35-dependent case speak for itself as the "why" — not the "what."

Your actual differentiator isn't fraud detection. It's analyst productivity. The copilot, the unified queue, the automated checklist, the one-click eligibility check replacing three systems — that's where Prudential feels pain every day, not just when fraud happens. Lead with that. The fraud detection is the exciting headline; the daily workflow improvement is the reason they'll actually buy it.

The Reframe You Actually Need

First: You're Not Wrong

Prudential's fraud detection being simplistic at the examiner level is probably accurate. Most supplemental health operations rely on:

- Human eyeballs catching obvious things
- Basic system edits (date checks, eligibility flags)
- Hoping the auto-adjudicator's simple rules are enough

That IS the opportunity. But let me be direct about where your thinking needs to shift.

The Thing You're Missing

You keep framing this as a **fraud detection** problem. It's not — at least not primarily. It's an **examiner workflow** problem that includes fraud detection as one component.

Here's why that matters:

Fraud detection without real data = a demo that smart people see through

Examiner workflow transformation with fraud detection built in = something you can prove with synthetic data because the value is in the process, not the predictions

Think about what an examiner actually does all day:

Morning: Open queue. 20-25 claims waiting.

For EACH claim:

1. Open claim in system A
2. Check eligibility in system B
3. Check policy details in system C
4. Read medical documents (if attached)
5. Mentally check: does anything look wrong?
6. Check their OneNote for rules they need to remember
7. Check if there's a PMR or CBR task
8. Make a decision
9. Document it
10. Move to next claim

Time per clean claim: 10-15 minutes

Time per complex claim: 30-60 minutes

Time per suspicious claim: hours + escalation paperwork

Your platform collapses steps 1-6 into one screen and one conversation. That's the value you can prove without real data. The fraud detection layer is what makes it *exciting*, but the workflow consolidation is what makes it *useful every single day*.

The Right Framing

Stop saying this:

“We built AI-powered fraud detection for supplemental health claims”

Start saying this:

“We built an AI copilot that helps examiners work every claim faster and catches fraud patterns that manual review can't see at scale”

The difference:

"Fraud Detection" Framing	"Examiner Copilot" Framing
Value depends on catching real fraud	Value exists on every single claim
Needs real data to prove	Proves itself with workflow demonstration
Used occasionally (when fraud happens)	Used all day every day
Hard to measure ROI until fraud is caught	Easy to measure: time per claim drops
Competes with FICO, SAS, Shift Technology	Competes with... nothing. They don't have this
Prudential asks "does it actually detect fraud?"	Prudential asks "can we pilot this with a team?"

What You Can Sell Without Real Data

Sell the workflow, demonstrate the detection

Things you CAN prove with synthetic data:

1. **Unified interface** — One screen instead of three systems. You can show this works regardless of whether the data is real or synthetic. The examiner sees the value immediately.
2. **Checklist automation** — Their OneNote checklist turned into tracked, documented steps. Every examiner does this manually today. You automate the tracking, not the judgment.
3. **Context surfacing** — When an examiner opens a claim, everything relevant is already there: policy details, family history, open tasks, risk flags. Today they hunt for this across systems.
4. **Documentation** — Every investigation step is logged. Today it's manual notes. Your system creates an audit trail automatically.
5. **Escalation packages** — Instead of manually writing up a case for SIU, click a button. This works whether the underlying fraud is real or synthetic.

Things you can DEMONSTRATE but not PROVE with synthetic data:

6. **Rule-based detection** — You can show the framework catches patterns. You can't prove the thresholds are right. That's fine — say "these thresholds are configurable and would be calibrated with your historical data."
7. **Risk scoring** — You can show the scoring framework and explain what it weighs. You can't prove the scores are accurate. Say "Phase 1 is heuristic, Phase 2 trains on your examiners' actual decisions."
8. **Network analysis** — You can show how a dependent ring WOULD appear in the graph. You can't prove these exist in their data. Say "this is the detection capability — your data tells us what's actually there."
9. **Document analysis** — You can show what checks would run. You can't run them on real documents. Say "simulated in prototype, production version integrates document forensics."

What You Should Actually Be Honest About

In your demo or pitch, have a slide or moment that says something like:

"This prototype runs on synthetic data designed to demonstrate the platform's capabilities. We embedded realistic fraud scenarios based on patterns Prudential has encountered. The architecture is production-ready — the data calibration happens during implementation when we connect to your actual claims systems. What we're showing you today is how your examiners would work, not a tuned fraud model."

This is **more credible** than pretending synthetic results prove anything. Decision-makers respect honesty and distrust vendors who oversell.

Your Actual Competitive Advantage

You said the prototype wasn't originally built for Prudential. That's actually a strength if you frame it right:

"This platform was built domain-agnostic. We configured it for supplemental health based on our discovery sessions. The same architecture works across lines of business. That means Prudential doesn't just get a tool for supplemental health — they get a platform that scales to any claims operation."

What to Do About Your Knowledge Gap

You said you have zero idea how fraud analysis works. Here's what you need to know that's actually relevant to building this:

Fraud detection is not magic. It's three things:

1. RULES
"If X happens, flag it"
You have 14 of these. They're reasonable guesses.
In production, you'd have hundreds, calibrated by data.
Your job: build the engine that runs any rule.
NOT your job: know every rule.
2. PATTERNS
"This combination of normal things is abnormal"
Risk scoring and network analysis.
You can build the framework.
The patterns come from data you don't have yet.
Your job: build the framework that finds patterns.
NOT your job: know what patterns exist at Prudential.
3. INVESTIGATION
"Here's why this is suspicious, here's the evidence"
This is where your copilot shines.
This is the part examiners do manually today.
Your job: make investigation faster and documented.
THIS is your job and you can do it well.

The thing most fraud detection systems get wrong is they focus on #1 and #2 and ignore #3. They flag stuff and dump it on an analyst with no context. Your copilot IS the differentiator.

Practical Next Steps

Stop trying to:

- Pretend you have data you don't have
- Build a production fraud detection model
- Compete with established fraud detection vendors
- Know everything about fraud analysis

Start doing:

- Build the examiner workflow you outlined (it's good)
- Make the copilot genuinely useful for daily claim work
- Show the fraud detection framework as extensible architecture
- Be honest about what's synthetic and what's real
- Ask for a second KT session focused specifically on examiner daily workflow (not fraud — workflow)

The ask for Prudential:

"We'd like to observe an examiner working claims for 2 hours. Not their fraud cases — their normal Tuesday morning. We want to see what systems they touch, what's slow, what's manual, what's annoying. That's what we're solving first. The fraud layer sits on top."

If you get that session, you'll build something 10x more useful than another fraud model nobody trusts because it wasn't trained on real data.

Bottom Line

Your plan is good. Your architecture is sound. Your gap isn't technical — it's positioning. You're selling a fraud detection system when you should be selling an examiner productivity platform with embedded fraud intelligence. One requires real data to prove. The other proves itself the moment an examiner touches it.